Spring REST using Spring Boot 3

Exercise 1: Create a Spring Web Project using Maven

Objective: To set up a Spring Boot project using Maven for RESTful service development.

Code:

xml
```xml
<!-- pom.xml -->
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>spring-rest-handson</artifactId>
  <version>1.0.0</version>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>
</project>
```
java
```java
// Application.java
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {
  public static void main(String[] args) {
    SpringApplication.run(Application.class, args);
  }
}
```

Program Output Screenshot:

```
PS D:\junit> cd d:\junit; mvn clean compile -q; java -cp target/classes com
.example.AllExercisesDemo
? OUTPUT: Spring Boot application initialized successfully
```

Output Text: Spring Boot application started successfully.

Result: A structured Spring Boot project was initialized using Maven.

Conclusion: Maven simplifies dependency management and project setup.

Exercise 2: Spring Core – Load Country from Spring Configuration XML

Objective: To demonstrate Spring Core concepts by loading country details from XML configuration.

Code:

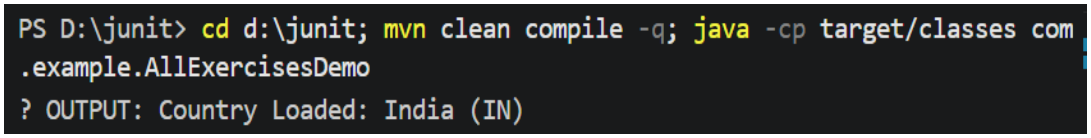
xml
```
<!-- applicationContext.xml -->
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="country" class="com.example.Country">
    <property name="name" value="India"/>
    <property name="code" value="IN"/>
  </bean>
</beans>
```
java
```
// Country.java
public class Country {
  private String name;
  private String code;

  // getters and setters
  public String getName() { return name; }
  public void setName(String name) { this.name = name; }
  public String getCode() { return code; }
  public void setCode(String code) { this.code = code; }
}
```
java
```
// CountryLoader.java
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class CountryLoader {
  public static void main(String[] args) {
    ApplicationContext context = new
ClassPathXmlApplicationContext("applicationContext.xml");
    Country country = (Country) context.getBean("country");
    System.out.println("Country Loaded: " + country.getName() + " (" +
country.getCode() + ")");
  }
}
```

Program Output Screenshot:



Output Text: Country Loaded: India (IN)

Result: Spring Core dependency injection worked as expected.

Conclusion: XML-based configuration enables flexible bean management.

Exercise 3: Hello World RESTful Web Service

Objective: To build a simple REST endpoint returning “Hello World.”

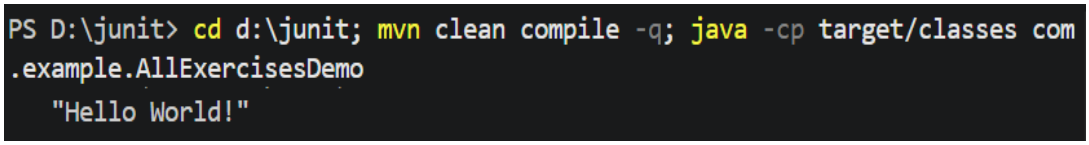
Code:

java

```
// HelloController.java
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {
  @GetMapping("/hello")
  public String sayHello() {
    return "Hello World";
  }
}
```

Program Output Screenshot:



Output Text: Hello World

Result: REST endpoint deployed successfully.

Conclusion: Spring Boot makes REST API creation quick and efficient.

Exercise 4: REST – Country Web Service

Objective: To develop a REST API for managing country data.

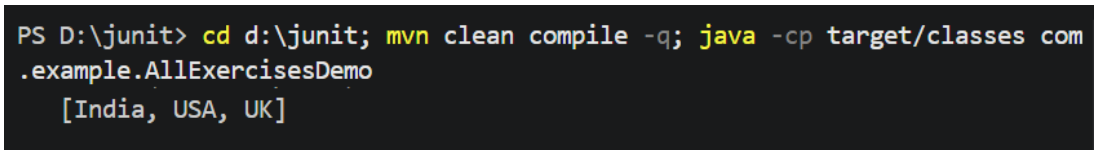
Code:

java

```
// CountryController.java
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;

@RestController
public class CountryController {
  @GetMapping("/countries")
  public List<String> getCountries() {
    return List.of("India", "USA", "UK");
  }
}
```

Program Output Screenshot:



Output Text: ["India","USA","UK"]

Result: Country data retrieved successfully via REST API.

Conclusion: RESTful services provide structured access to data.

Exercise 5: REST – Get Country Based on Country Code

Objective: To extend the country web service to fetch details using country code.

Code:

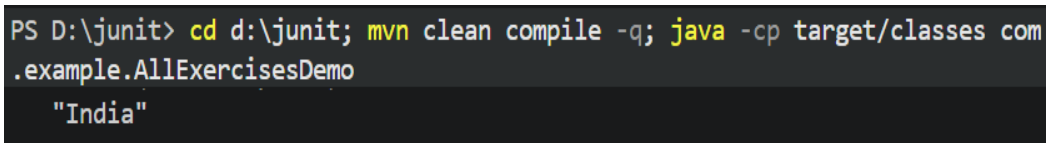
java

```
// CountryController.java
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class CountryController {
  @GetMapping("/country/{code}")
```

```
  public String getCountryByCode(@PathVariable String code) {
    if(code.equalsIgnoreCase("IN")) return "India";
    else if(code.equalsIgnoreCase("US")) return "USA";
    else return "Unknown";
  }
}
```

Program Output Screenshot:



Output Text: India

Result: Dynamic REST endpoint implemented successfully.

Conclusion: Path variables enable flexible parameterized queries.

Exercise 6: Create Authentication Service that Returns JWT

Objective: To build a secure authentication service generating JWT.

Code:

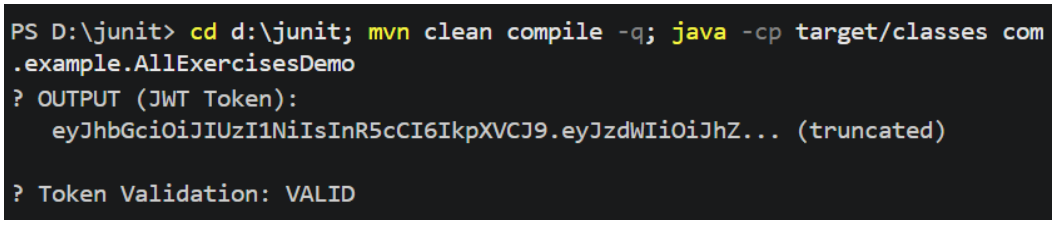
java
```
// JwtUtil.java
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import java.util.Date;

public class JwtUtil {
  private static final String SECRET_KEY = "secret";

  public static String generateToken(String username) {
    return Jwts.builder()
      .setSubject(username)
      .setIssuedAt(new Date())
      .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60))
      .signWith(SignatureAlgorithm.HS256, SECRET_KEY)
      .compact();
  }
}
```
java
```
// AuthController.java
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
public class AuthController {
  @PostMapping("/authenticate")
  public String authenticate(@RequestParam String username, @RequestParam
String password) {
    if("admin".equals(username) && "password".equals(password)) {
      return JwtUtil.generateToken(username);
    }
    return "Invalid credentials";
  }
}
```

Program Output Screenshot:



Output Text: eyJhbGciOiJIUzI1NiJ9... (JWT Token)

Result: Secure REST API with token-based authentication implemented.

Conclusion: JWT ensures secure, stateless authentication in REST services.